Project 59: Monte Carlo Sales Forecast - Source Code

```
"""
Project 59: Monte Carlo Sales Forecast
Concepts: random and statistics modules
Simulates thousands of possible monthly sales outcomes to estimate
expected revenue and risk.
"""

import random
import statistics


def simulate_month(avg_units, units_std, avg_price, price_std):
    """Simulate ONE month: random units sold x random selling price."""
    units = max(0, random.gauss(avg_units, units_std))
    price = max(0, random.gauss(avg_price, price_std))
    return units * price


def run_simulation(n, avg_units, units_std, avg_price, price_std):
    """Run n simulations and return a list of revenues."""
    return [simulate_month(avg_units, units_std, avg_price, price_std)
            for _ in range(n)]


def percentile(data, p):
    """Return the p-th percentile of a list (0-100)."""
    data = sorted(data)
    index = int(p / 100 * (len(data) - 1))
    return data[index]


def text_histogram(data, bins=10, width=40):
    """Print a simple text histogram of outcomes."""
    low, high = min(data), max(data)
    step = (high - low) / bins
    counts = [0] * bins
    for x in data:
        i = min(int((x - low) / step), bins - 1)
        counts[i] += 1
    biggest = max(counts)
    print("\nDistribution of monthly revenue:")
    for i, c in enumerate(counts):
        start = low + i * step
        bar = "#" * int(c / biggest * width)
        print(f"{start:>10,.0f} | {bar} {c}")


def get_float(prompt, default):
    """Read a number from the user, falling back to a default."""
    while True:
        value = input(f"{prompt} [{default}]: ").strip()
        if not value:
            return default
        try:
            return float(value)
        except ValueError:
            print("Please enter a valid number.")


def main():
    print("=== Monte Carlo Sales Forecast ===")
    avg_units = get_float("Expected units sold per month", 500)
    units_std = get_float("Units uncertainty (std dev)", 80)
```

```
    avg_price = get_float("Average selling price (Rs.)", 200)
    price_std = get_float("Price uncertainty (std dev)", 15)
    target = get_float("Monthly revenue target (Rs.)", 100000)
    n = int(get_float("Number of simulations", 10000))

    results = run_simulation(n, avg_units, units_std, avg_price, price_std)

    mean = statistics.mean(results)
    median = statistics.median(results)
    stdev = statistics.stdev(results)
    p5 = percentile(results, 5)
    p95 = percentile(results, 95)
    prob_target = sum(r >= target for r in results) / n * 100

    print(f"\n--- Results from {n:,} simulations ---")
    print(f"Expected revenue (mean) : Rs. {mean:,.0f}")
    print(f"Median revenue          : Rs. {median:,.0f}")
    print(f"Risk (std deviation)    : Rs. {stdev:,.0f}")
    print(f"Worst case (5th pct)    : Rs. {p5:,.0f}")
    print(f"Best case (95th pct)    : Rs. {p95:,.0f}")
    print(f"90% range               : Rs. {p5:,.0f} to Rs. {p95:,.0f}")
    print(f"Chance of hitting Rs. {target:,.0f} : {prob_target:.1f}%")

    text_histogram(results)


if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print(f"\nError: {e}")
    input("\nPress Enter to exit...")
```